TAB: technical brief

Spending limits for AI agents, paid in USDC on Solana over x402. Prepared 8 Oct 2026 for internal review. Live product: tab-agent-one.vercel.app (Solana mainnet). This document is self-contained so it can be read by a person or uploaded to an AI assistant.

How to use this document with an AI assistant

Upload this file and ask questions such as: "Explain TAB to a non-technical investor", "What are the main risks before launch?", "Walk me through what happens when an agent pays for something", or "How does TAB make money and what does it cost to run?". Every number and behaviour below describes the system as built, not a plan, unless it is marked as pending.

---

1. The product in one paragraph

AI agents increasingly need to pay for things on their own: API calls, model inference, data, scraping, one request at a time. Today the only way to let an agent pay with crypto is to give it a wallet, which means giving it the private key and everything in that wallet. **TAB gives an agent a "tab" instead: a USDC allowance with a hard limit, a maximum per payment, an expiry date and an optional list of approved services.** The agent gets a key that lets it ask TAB to pay, never a wallet key. The person who opened the tab can revoke it at any moment, and whatever is unspent goes back to their wallet.

Key terms

TERM	MEANING
Tab	One spending allowance: limit, max per payment, expiry, optional allowlist. A user can have many.
Owner	The human (or company) who opens and funds the tab with their own Solana wallet.
Agent	The AI program that spends from the tab. It holds only an agent key (credential).
Agent key	A random secret of the form `tab_live_…`, shown once, stored by TAB only as a keyed hash. Works for exactly one tab.
x402	An open standard for paying for web requests: a service answers HTTP 402 "Payment Required" with a price, the client pays and retries. TAB implements x402 version 2, "exact" scheme on Solana.
Facilitator	Part of x402. The service's helper that checks a signed payment, pays the Solana network fee and submits it to the chain.
Delegate / allowance	A native feature of Solana's token program: an account owner can approve another key to move up to N tokens from that account. TAB's key is that "delegate".
USDC	The dollar stablecoin. Amounts are handled in base units: 1 USDC = 1,000,000 units.

2. Why it is non-custodial

TAB never holds anyone's money. This was a founding decision and the whole design follows from it.

1. When the owner opens a tab, their wallet signs **one Solana transaction** that creates a brand-new token account **owned by the owner's wallet**, moves the tab's limit into it, and approves TAB's delegate key to spend **up to exactly the limit** from it.
2. The money sits in the owner's own account. TAB only holds a permission to spend from it, and that permission is enforced by the Solana token program itself, not by TAB's database. Even if TAB's database had a bug, the chain would refuse to move more than the approved amount.
3. The owner can revoke that permission or close the account **directly from their wallet, without TAB**, even if TAB's servers are down.
4. Each tab gets its own account because a Solana token account can only have one delegate and one allowance at a time.

> **What TAB can and cannot do**
>
> **Can:** sign payments from a tab up to what is left of its limit; refuse any payment that breaks a rule; stop signing the moment the owner revokes.
> **Cannot:** touch anything else in the owner's wallet; spend past the limit (the token program blocks it on chain); stop the owner revoking or closing from their wallet.

This was proven before building anything: a "spike" on Solana devnet on 7 Oct 2026 showed that a delegate-signed x402 payment is accepted, settled and returns the paid resource; that a payment above the remaining allowance is rejected before anything moves; and that after the owner revokes, payments fail and nothing moves.

---

3. How it works, step by step

3.1 Opening a tab (owner)

1. The owner connects a Solana wallet (Phantom, Solflare, Backpack or any Wallet Standard wallet) on the dashboard and signs in by **signing a text message** (Sign-In With Solana). Signing in costs nothing and moves nothing.
2. They choose: a limit (1 to 1,000 USDC during beta), a max per payment (or none), an expiry (1 hour, 1 day, 7 days, 30 days) and optionally an allowlist of service payment addresses.
3. TAB's server builds the transaction and the wallet shows it for approval. It contains, in one signature: the **0.20 USDC opening fee** to TAB's treasury, creating the tab's token account (owner = the user), moving the limit into it, and approving TAB's delegate for exactly the limit.
4. After it confirms on chain, TAB verifies the account on chain and shows the agent key **once**. The owner gives this key to their agent.

3.2 An agent paying for something

1. The agent calls a paid service through TAB's SDK (`tab.fetch(url)` , a drop-in for normal `fetch`).

2. The service answers **402 Payment Required** with its price, token and payment address in a `PAYMENT-REQUIRED` header.
3. The SDK refuses locally if the price is above the agent's own client-side ceiling (`maxPayment`). Otherwise it sends the bill to TAB: `POST /v1/pay` with the agent key and a unique idempotency key.
4. TAB runs **five checks, in this order**. Any failure returns a typed error and **nothing is signed**:
    1. The tab is open (not revoked, closed or unfunded). Error: `TAB_REVOKED` / `TAB_CLOSED` / `TAB_NOT_FOUND` .
    2. It hasn't expired. Error: `TAB_EXPIRED` .
    3. The amount is within the max per payment. Error: `PAYMENT_EXCEEDS_TAB_LIMIT` .
    4. The service is on the allowlist, if one is set. Error: `RECIPIENT_NOT_ALLOWED` .
    5. There is room left: spent + in-flight + this payment ≤ limit. Error: `INSUFFICIENT_TAB_BALANCE` .
5. If all pass, TAB **reserves** the amount (so concurrent payments cannot share the same balance), signs a USDC transfer from the tab account with its delegate key, and returns the signed payment to the SDK.
6. The SDK retries the request with a `PAYMENT-SIGNATURE` header. The service's facilitator adds its own signature as fee payer, submits it, and the service returns the paid content (HTTP 200).
7. TAB marks the payment **settled only when it finds the transaction carrying its signature confirmed on chain**.

3.3 Closing out

The owner clicks Revoke. TAB stops signing immediately, then the owner's wallet signs one transaction that removes the permission, moves the unspent USDC back to their wallet and closes the account, which also returns the account rent (about 0.002 SOL).

---

4. Money safety guarantees (the invariants)

These are hard rules in the code, each backed by automated tests.

GUARANTEE	HOW IT IS ENFORCED
An agent can never spend more than the tab has left.	Database transaction with a row lock on the tab (`SELECT … FOR UPDATE`) plus a database constraint that spent + reserved ≤ limit. Independently, the on-chain allowance is a second ceiling TAB cannot override.
Two simultaneous payments can't spend the same balance.	Reserve before signing, settle after. Tested: 50 parallel payments against a tab with room for 5 → exactly 5 succeed. The test was confirmed to fail if the row lock is removed.
A repeated request never pays twice.	Every payment carries an idempotency key; a repeat returns the original signed payment, which can only land on chain once.
A crash mid-payment never loses or double-spends money.	Tested by killing the process between reserve and settle. On restart, a held payment is settled if its transaction is on chain, and released only after its blockhash has expired (after which it can no longer land).
Amounts are exact.	All money is integer base units (bigint). No floating point anywhere.
Agent keys can't be stolen from the database.	Random, shown once, stored only as a keyed hash, scoped to one tab, rotatable.
TAB's signing key is never exposed.	Loaded only from a server secret, never logged or returned by any endpoint.
Unsafe tokens are refused.	Token-2022 mints with permanent delegate, transfer hook, transfer fee, non-transferable, confidential, default-frozen or pausable extensions are rejected. V1 accepts USDC only.

Tests: 34 automated tests run against a real Postgres database: concurrency, crash injection, replay, and every error code. A full devnet end-to-end run (sign in, open, agent pays, cap refusal, revoke, reclaim, close) was done through both a script and the real dashboard, with the database matching the chain after every run.

5. Business model and running costs

ITEM	AMOUNT	WHO PAYS
Opening a tab	0.20 USDC flat, per tab	Owner, to TAB's treasury, inside the same signature that opens the tab (shown before signing)
TAB fee per payment	0	None
Solana network fee per payment	~0.00001 SOL	The service's x402 facilitator, not TAB and not the owner
Account rent per tab	~0.002 SOL	Owner, refunded in full when the tab is closed
Network fee to open/close a tab	A fraction of a cent in SOL	Owner's wallet
Hosting (web app + API)	Free tier	Vercel Hobby
Database	Free tier	Neon Postgres
Solana RPC	Free public endpoint	None (see risks)
Live demo on the site	~0.00001 SOL per demo payment	TAB's small demo fee-payer wallet; the demo pauses itself when that wallet runs low

TAB therefore has **no per-payment cost**: revenue is 0.20 USDC per tab opened, and the marginal cost of a tab to TAB is effectively zero. Treasury address (public): `3VjaxvebqEA8J6gCLLwU6Vu1jYdNQH4cYKFMiPN8cBmH`.

6. For developers: integrating an agent

```
npm i @tabpay/sdk

import { Tab } from "@tabpay/sdk";

const tab = new Tab({
  credential: process.env.TAB_CREDENTIAL,  // the agent key, tab_live_…
  maxPayment: "500000",                    // client-side ceiling: 0.50 USDC
});

const res = await tab.fetch("https://api.example.com/forecast");
// 402 → checked → signed → paid → 200
```

Free requests pass straight through untouched. Paid ones are handled automatically within the tab's rules. Rejections throw an error with one of the codes above.

HTTP API (summary)

ENDPOINT	USED BY	PURPOSE
`POST /v1/pay`	Agent (bearer agent key)	Check a bill against the tab's rules and return a signed payment
`POST /v1/payments/:id/confirm`	Agent	Report the transaction id so TAB settles it immediately (verified on chain)
`GET /v1/tab`	Agent	Status, limit, spent, in flight, remaining, cap, expiry
`POST /v1/auth/nonce`, `/verify`, `/logout`	Dashboard	Sign-In With Solana (message signature) and session cookie
`GET/POST /v1/tabs`, `/v1/tabs/:id`	Dashboard (owner)	List, create (returns the unsigned transaction), view
`/v1/tabs/:id/activate | rotate | revoke | close`	Dashboard (owner)	Activate after funding, new agent key, stop signing, return funds and close

7. Architecture

LAYER	TECHNOLOGY
Web app (landing, docs, dashboard)	React + Vite, served by Vercel. Wallets via Wallet Standard. Sign-in is a message signature only.
API	TypeScript, Hono framework, running as a Vercel serverless function
Database	PostgreSQL on Neon (pooled connections). Tables: users, tabs, allowed recipients, payments, auth nonces.
Blockchain	Solana mainnet, SPL Token program, USDC (mint `EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v`), libraries @solana/web3.js and @solana/spl-token
Payments protocol	x402 version 2, "exact" scheme on Solana (official @x402 packages)
Agent SDK	TypeScript package `@tabpay/sdk` (in the repository; not yet published to npm)
Background jobs	A reconcile job settles or releases stuck payments from chain evidence. It runs on every read of a tab and once a day by scheduled job.
Source code	Private GitHub repository

Payment lifecycle states

`RESERVED` (amount held) → `DISPATCHING` (signed, handed to the agent) → `SETTLED` (found confirmed on chain) or `RELEASED` (blockhash expired, never landed; the hold is returned). `AMBIGUOUS` is used when the chain could not be read; it is retried and never released early.

The chain is the only source of truth. A service can sometimes land a payment and still answer 402 (if its own facilitator is slow); TAB then counts it as spent, because it was, and the SDK retries once with the same signed payment, which cannot be charged twice.

8. Current status (8 Oct 2026)

Done

- Full product built and deployed on **Solana mainnet**: landing page, docs, dashboard (open, view, revoke, reclaim, rotate key), agent API, SDK.
- Non-custodial design proven on devnet; mainnet tab-opening transaction (all instructions including the fee) simulated successfully against real mainnet state.
- 0.20 USDC opening fee wired to the treasury.

- Beta safety cap: a tab can hold at most 1,000 USDC.
- 34 automated tests passing, including concurrency and crash recovery.
- Site redesigned with interactive explainers (live simulated agent, five-checks demo, non-custodial demo).

Pending (owner actions)

- One real-money mainnet run: open a ~1.2 USDC tab, let the test agent pay, revoke and reclaim.
- Fund the demo fee-payer wallet with ~0.005 SOL so the live demo can run on mainnet.
- Custom domain and X (Twitter) handle.
- Publish the SDK to npm (package name is a placeholder until then).

---

9. Risks and open decisions

Be aware before scaling

- **No third-party security audit yet.** Mitigated by the on-chain allowance (TAB cannot exceed it even if its code is wrong), the 1,000 USDC per-tab beta cap, and the test suite, but an audit is the standard next step before large balances.
- **TAB's delegate signing key lives in the hosting provider's encrypted environment variables, not a hardware key service (KMS).** This was a deliberate owner decision for launch. If that key leaked, an attacker could spend from open tabs up to each tab's remaining limit (never more, never from the rest of anyone's wallet). Owners can revoke from their wallets at any time. Moving to KMS is an available upgrade.
- **Free public Solana RPC.** It is rate-limited; under load, confirmations can be slow. Payments are never lost (they settle from chain evidence later), but the dashboard may lag. A paid RPC is a cheap upgrade when volume justifies it.
- **If TAB is down, agents cannot pay** until it is back. Funds are unaffected and owners keep full control from their wallets.
- **Daily scheduled reconcile** (Vercel free tier limit). Stuck payments also reconcile whenever a tab is viewed, so this only matters for tabs nobody looks at.
- **Ecosystem dependency:** TAB works with services that accept x402 V2 payments in USDC on Solana. Its reach grows with x402 adoption.

---

10. Out of scope for version 1

Token swaps, assets other than USDC, other blockchains, payment channels, a marketplace, a TAB token, fiat on-ramps, DAO/governance, staking. These were excluded deliberately to ship a small, safe core.

11. Frequently asked questions

Can TAB run off with users' money?

No. Funds sit in accounts owned by the users' wallets. TAB can only move funds from an open tab, to a service's payment address, through the five checks, and never more than what remains of that tab's limit, which the Solana token program enforces.

What stops a buggy or hijacked agent from draining a tab?

The max per payment stops one huge charge; the allowlist stops payments to unknown services; the limit caps the total; the expiry ends it automatically; and the owner can revoke instantly. The worst case is losing what remains on that one tab, which the owner chose in advance.

Why would a user pay 0.20 USDC?

Because the alternative is giving an agent a full wallet key. 0.20 USDC buys a scoped, expiring, revocable allowance with a payment ledger, with no per-payment fees.

Does TAB need its own payment network or token?

No. It uses USDC, Solana's standard token program and the open x402 protocol. Services don't need to integrate anything TAB-specific; if they accept x402 on Solana, TAB-funded agents can pay them.